JAVA TESTING GUIDE

1. What is Test-Driven Development (TDD)?

Test-Driven Development (TDD) is a software development approach where you write a failing test **BEFORE** you write any production code. This might sound backwards at first — but it is one of the most powerful techniques for writing clean, reliable, and bug-free Java programs.

Think of TDD like this: before building a bridge, an engineer specifies exactly how much weight it must hold. The test (specification) comes first — the construction (code) comes second.

1.1 The Red-Green-Refactor Cycle

Every TDD cycle follows exactly three steps:

Step	What You Do	Meaning
RED	Write a test that FAILS	Your code doesn't do this yet — and that's OK
GREEN	Write the MINIMUM code to make the test pass	Just enough code — nothing more
REFACTOR	Clean up your code WITHOUT breaking the test	Improve design while keeping tests green

TIP You repeat this cycle for every small feature. Many short cycles are better than one big one.

1.2 Why Use TDD?

As a beginner, TDD gives you these benefits:

- You always know when your code is working correctly
- You catch bugs immediately, before they become hard to find
- You write smaller, simpler methods because tests push you to break things down
- Your tests become documentation — they explain what the code is supposed to do

Refactoring (improving code structure) becomes safe because tests catch regressions

NOTE TDD does NOT mean 'write tests after your code is done'. Tests come FIRST, always.

2. What You Need Before You Start

2.1 Java Installation

This is installed already.

2.2 Your Text Editor

You do NOT need an IDE for now. You have your notepad and text editor

3. Required JAR Files — JUnit 5

JUnit is the testing framework you will use. Since you are not using Maven or Gradle, you need to download JAR files manually. A JAR (Java ARchive) is a packaged library of compiled Java code.

3.1 Which JAR Files to Download

You need exactly these files. Download all of them from Maven Central Repository (https://repo1.maven.org/maven2):

JAR File Name	Purpose	Download Path on Maven Central
junit-platform-console-standalone-1.11.0.jar	ALL-IN-ONE jar — runs JUnit 5 tests from the command line. This single file replaces all others.	org/junit/platform/junit-platform-console-standalone/1.11.0/

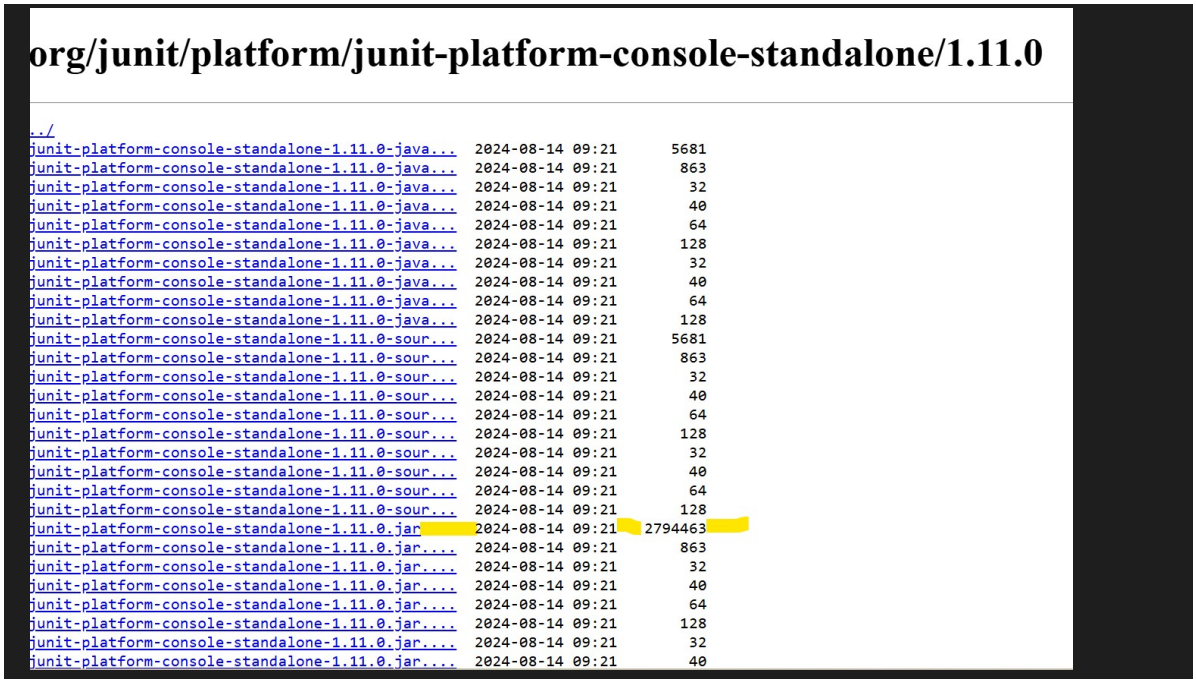
> **NOTE** You only need ONE jar file. The `junit-platform-console-standalone` JAR bundles JUnit 5, JUnit Jupiter API, JUnit Engine, and Hamcrest all together. Download just this one file.

3.2 Step-by-Step Download Instructions

Windows — Downloading the JAR

1. Open your web browser and go to:

```
https://repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone/1.11.0/
```



2. Click on the file named(as I highlighted for you):

```
junit-platform-console-standalone-1.11.0.jar
```

3. Save it to a folder you will remember.

Linux — Downloading the JAR

1. Open your terminal
2. Create a folder and download with wget:

```
wget https://repo1.maven.org/maven2/org/junit/platform/junit-platform-
console-standalone/1.11.0/junit-platform-console-standalone-1.11.0.jar
```

You can also just follow the instructions for the windows folks

3.3 Recommended Project Folder Structure

Remember that the jar file must be in the folder before your test file and code file.

Also watch the version of the jar file - I use 1.10.2. If you follow this guide, you should be using 1.11.0

4. Terminal Commands — Compiling & Running Tests

WARNING The commands below assume your terminal is **already inside your project folder**. Always cd into your project folder first.

4.1 Navigate to Your Project

Windows (Command Prompt)	Linux (Terminal)
`cd C:\javaTasks\tdd`	`cd ~/javaTasks/tdd`

4.2 Compile Actual Code

This compiles your Java files in the folder and puts the compiled .class files in a `out/` folder:

For all files in the folder

Windows	Linux
`javac -d out *.java`	`javac -d out *.java`

For one file

Windows	Linux
`javac -d out Kata.java`	`javac -d out Kata.java`

4.3 Compile Test Code

This compiles your test files (they need the JAR and your classes on the classpath):

Windows	Linux
`javac -cp "junit-platform-console-standalone-1.11.0.jar;out" -d out TestFile.java CodeFile.java`	`javac -cp "junit-platform-console-standalone-1.11.0.jar:out" -d out TestFile.java CodeFile.java`

TIP Windows uses semicolons (`;`) to separate classpath entries. Linux uses colons (`:`). This is the most common beginner mistake!

4.4 Run the Tests

This launches the JUnit console runner and executes all your tests:

Windows:

```
java -cp "junit-platform-console-standalone-1.11.0.jar;out" org.junit.platform.console.Console
Launcher --scan-class-path
```

Linux:

```
java -cp "junit-platform-console-standalone-1.11.0.jar:out" org.junit.platform.console.Console
Launcher --scan-class-path
```

4.5 Reading Test Output

After running tests, you will see output like this:

```
|
└─ JUnit Jupiter ✓
   └─ CalculatorTest ✓
      ├─ testThatIAdd2NumbersResultIsNumberOnePlusNumberTwo() ✓
      ├─ testThatISubtractTwoNumbersDifferenceIsNumberOneMinusNumberTwo() ✓
      └─ testDivideByZeroArithmeticExceptionIsThrown()✓


Test run finished after 45 ms
[         3 tests found      ]
[         0 tests aborted    ]
[         3 tests started    ]
[         0 tests skipped    ]
[         3 tests successful ]
[         0 tests failed     ]
```

Symbol / Word	What It Means
✓ (green tick)	Test passed — your code works for this case
✗ or FAILED	Test failed — your code does not do what the test expects
tests found	Total number of test methods discovered
tests successful	Number that passed — aim for this to equal 'tests found'
tests failed	Number that failed — read the error message to fix your code

5. TDD Example — A Calculator

Let us walk through a complete TDD example from scratch. We will build a simple Calculator class using the Red-Green-Refactor cycle.

5.1 Set Up the Project Folder

5.2 RED — Write a Failing Test First

Create the file `CalculatorTest.java` and type:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    @Test
    public void testThatIAdd2NumbersResultIsNumberOnePlusNumberTwo() {
        Calculator calc = new Calculator();
        int result = Calculator.add(3, 4);
          int expected = 7;
        assertEquals(expected, result);
    }
}
```

Now try to compile the test — it will FAIL because Calculator does not exist yet. That is the RED step.

```
javac -cp "junit-platform-console-standalone-1.11.0.jar:out" -d out CalculatorTest.java
```

You will see an error like:

```
error: cannot find symbol
       Calculator calc = new Calculator();
```

NOTE This error is EXPECTED and GOOD! It means your test is ahead of your code — exactly where it should be in TDD.

5.3 GREEN — Write Just Enough Code to Pass

Create the file `Calculator.java` and type the MINIMUM code needed:

```
public class Calculator {


    public int add(int a, int b) {
        return a + b;
    }
}
```

Now compile and run:

```
javac -cp "junit-platform-console-standalone-1.11.0.jar:out" -d out CalculatorTest.java
Calculator.java
java  -cp "junit-platform-console-standalone-1.11.0.jar:out" org.junit.platform.console.Console
Launcher --scan-class-path
```

You should see:

```
[         1 tests successful ]
[         0 tests failed     ]
```

NOTE Congratulations — you are GREEN! The test passes. Do not add any extra code yet.

5.4 REFACTOR — Clean Up

In this case the code is already clean and simple — no refactoring needed. Move on to the next test.

5.5 Repeat the Cycle — Add More Features

Add a subtraction test to `CalculatorTest.java`:

```
    @Test
    public void testThatISubtractTwoNumbersDifferenceIsNumberOneMinusNumberTwo() {
        Calculator calc = new Calculator();
        int difference = calc.subtract(9, 4);
        int expected = 5;
        assertEquals(expected , difference );
    }


    @Test
    public void testDivideByZeroArithmeticExceptionIsThrown() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(10, 0));
    }
```

Run the tests — they will be RED. Then add `subtract()` and `divide()` to Calculator.java:

```
    public int subtract(int a, int b) {
        return a - b;
    }

    public int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Cannot divide by zero");
        return a / b;
    }
```

Run the tests again — all 3 should be GREEN.

6. Common JUnit Assertions

All assertion methods come from the class: import static org.junit.jupiter.api.Assertions.*;

Assertion Method	What It Checks
assertEquals(expected, actual)	Checks that two values are equal (use for int, String, double, etc.)
assertNotEquals(unexpected, actual)	Checks that two values are NOT equal
assertTrue(condition)	Checks that a boolean expression is true
assertFalse(condition)	Checks that a boolean expression is false
assertNull(object)	Checks that an object reference is null
assertNotNull(object)	Checks that an object reference is NOT null
assertThrows(ExceptionType.class, () -> ...)	Checks that a specific exception is thrown
assertArrayEquals(expected[], actual[])	Checks that two arrays have the same contents

TIP When comparing doubles, always use: `assertEquals(3.14, result, 0.001)` — the third argument is the allowed tolerance (delta), because floating-point arithmetic is imprecise.

7. Common Errors & How to Fix Them

Error Message	What It Means & How to Fix It
cannot find symbol: class Calculator	You haven't created the class yet (RED step — this is normal). Create the file and recompile.
error: package org.junit.jupiter.api does not exist	The JUnit JAR is not on the classpath. Double-check the -cp path and make sure the JAR filename is exactly right.
';' expected or '{' expected	Syntax error in your Java file. Check for missing semicolons, brackets, or braces. Count opening { and closing }.
ClassNotFoundException: ...	Compiled class is not in the out/ folder. Run the javac -d out step again first.
0 tests found	Test methods must be public, void, have no parameters, and have the @Test annotation. Check your method signature.
Windows: The filename, directory name, or volume label syntax is incorrect	Check your path separators. Use backslash (\) on Windows, not forward slash.
Linux: No such file or directory	Check the path. Use ls to list files. Paths on Linux are case-sensitive: Calculator.java ≠ calculator.java

Website: www.semicolon.africa
Email: info@semicolon.africa
Tel: 0906 000 8609
Social: @semicolonafrica

semicolon

Have fun guys!